

CS253 Assignment 1

Memory-Efficient Versioned File Indexer Using C++

Student Details

Name: Rajasa

Roll Number: 240596

Objective

The objective of this assignment is to implement a memory-efficient word-level indexing system using C++. The program processes large text files and builds an index that maps each unique word to the number of times it appears in the file. The system supports analytical queries such as word count, top-K frequent words, and difference between two dataset versions.

Program Description

The program reads a dataset containing log records and creates a word-level frequency index using an `unordered_map`. Each word is converted to lowercase to ensure case-insensitive matching.

Example:

error → 2

debug → 1

info → 1

The index created by the program is then used to answer different types of queries.

Object-Oriented Design

The program is implemented using object-oriented programming with multiple classes.

Tokenizer

Converts words to lowercase for case-insensitive comparison.

BufferedReader

Reads the input file and extracts words.

Indexer

Builds the word-frequency index.

Query (Abstract Base Class)

Defines a virtual function used by all query types.

WordQuery

Returns the frequency of a specific word.

TopKQuery

Displays the top K most frequent words.

DiffQuery

Computes the difference in frequency of a word between two dataset versions.

Queries Implemented

Word Count Query

Returns the frequency of a given word in a dataset.

Top-K Query

Displays the top K most frequent words sorted in descending order.

Difference Query

Shows the difference in frequency of a word between two dataset versions.

Buffer Size

The program uses a fixed buffer size of 512 KB during execution.

Example Output:

Buffer Size: 512 KB

Execution Time

The execution time of the query is measured using the C++ chrono library and displayed after the query result.

Example:

Execution Time: 4.74 seconds

Compilation

```
g++ 240596_Rajasa.cpp -o analyzer
```

Execution

Word Query Example

```
.\analyzer --file test_logs.txt --version v1 --buffer 512 --query word --word error
```

Conclusion

The program successfully builds a word-level index from large text files and supports analytical queries such as word count, top-K frequent words, and difference queries. It also demonstrates object-oriented programming concepts including inheritance, polymorphism, templates, and exception handling.